Bug Report Classification with a Linear SVM and Enriched Text Features

Kanishk Suri, University of Birmingham, April 2026

1 Introduction

Bug reports are how users tell developers what is going wrong with a piece of software. The volume on large open-source projects is heavy: between the v3.0 and v3.1 releases of Eclipse the project received 13,016 bug reports in roughly a year, an average of 37 a day and a peak above 220 in a single day [1], and Mozilla was triaging around 300 reports a day in 2005. A human triager has to read each one and decide what category it is, who should look at it and how urgent it is, and as reports pile up this step becomes a real bottleneck. Mislabelled or ignored reports lead to delayed fixes and lower software quality [1].

This report looks at the first step of that pipeline: putting each incoming report into one of four categories (hardware, OS, platform, UI). I compare the Naive Bayes + TF-IDF classifier from Lab 1 [2] against a Linear SVM that uses word bigrams, character n-grams and a small set of hand-picked text statistics. Two datasets are used: a synthetic 3,000-report corpus generated from templates so that the comparison is fully controlled, and the computer-categories subset of 20 Newsgroups (3,918 real posts about hardware, Windows OS, Mac platform issues and X11 UI), which gives a noisier real-world view. Both classifiers are evaluated with stratified 10-fold cross-validation and a separate 80/20 held-out split, and the difference between them is checked with a two-tailed Wilcoxon signed-rank test [8] at alpha=0.05.

Classification was chosen because most later steps in bug management (assignment, duplicate detection, fix-time prediction) assume the report has already been correctly labelled [3], so a small accuracy gain at this point helps everything that follows.

2 Related Work

Antoniol et al. (2008) [13] is the early work in this area: they used Decision Trees, Naive Bayes and Logistic Regression on a TF-IDF representation of bug-report titles and descriptions to decide whether each report was actually a bug or an enhancement request, with text filtering, stemming and indexing as the preprocessing pipeline. Anvik et al. [1] used Naive Bayes for the related task of bug assignment in Eclipse and reached around 64% accuracy. Naive Bayes with TF-IDF is still the standard baseline because it is fast and handles sparse high-dimensional input without much tuning; its weakness is the conditional independence assumption, which means it cannot exploit the fact that words such as "kernel" and "panic" go together in OS-level reports.

Joachims [4] showed that SVMs tend to do better than Naive Bayes on text classification, and Limsettho et al. [2] saw the same effect on bug reports across several open-source projects. Tian et al. [3] (DRONE) added structured metadata (reporter history, component, version) to a regression model for priority prediction, which is evidence that non-textual fields help but also that they need a richer dataset than is publicly available for many projects. Zhang et al. [6] showed that character-level n-grams are useful for short noisy text because they pick up sub-word patterns (suffixes such as "Exception", "Fault" or "Panic") that word tokenisation misses.

More recent work moves to pre-trained language models. BERT [5] and similar transformer encoders usually push accuracy above linear models but need GPUs and a fine-tuning loop, which is not always practical for a small tool. Other work uses dense embeddings such as GloVe [10] and Word2Vec [11] with deeper architectures, for example the hierarchical attention network of Lam et al. [12]. These can give better accuracy but need pre-trained vectors and a more complex training setup. For a CPU-only coursework tool, sparse TF-IDF features keep the pipeline simple and easy to inspect, so I stayed with TF-IDF rather than moving to embeddings. Table 1 summarises the trade-offs across the families above.

Family	Example	Pros	Cons
NB + TF-IDF	[1, 2, 13]	Fast, low memory, handles sparse input well	Independence assumption; ignores word order
DT / LR + TF-IDF	[13]	Interpretable; LR gives	Lower accuracy than SVM

		probabilities	on text [4]
Linear SVM + TF-IDF	[2, 4]	Strong on high-dim text; margin objective	No probabilities; needs feature scaling
Linear SVM + word/char n-grams	this report, [6]	Catches sub-word patterns; tolerant of typos	Larger feature matrix; more memory
Embeddings + attention	[12]	Captures semantics; SOTA on hard datasets	Pre-trained vectors required; more compute
Pre-trained LM (BERT)	[5]	Best raw accuracy on noisy text	GPU and fine-tuning required

Table 1. Approaches to bug-report classification considered here.

3 Solution

3.1 Feature Engineering

The classifier sees three feature groups concatenated through scikit-learn FeatureUnion:

- **(1)** Word TF-IDF with bigrams. ngram_range=(1,2), sublinear TF scaling, max 50,000 features, min_df=2. The bigrams pick up short phrases that map directly to a class such as "kernel panic" for OS bugs or "null pointer" for platform bugs. Sublinear scaling tones down the very common tokens.
- **(2)** Character n-gram TF-IDF. analyzer='char_wb', ngram_range=(2,4), max 30,000 features, min_df=3. Character n-grams handle typos and pick up technical fragments (e.g. "BIOS", "Xorg", driver suffixes) that word tokenisation breaks apart.
- **(3)** Text statistics. Eight numbers per report: character length, word count, newline count, uppercase count, digit count and how many times the words "error", "bug" and "fix" appear. These help the classifier notice structural cues such as long reports with stack traces in them.

The three groups are stacked horizontally and then passed through a MaxAbsScaler so that the SVM optimiser is not dominated by the largest features.

3.2 Classifier

I use LinearSVC from scikit-learn with C=1.0, tolerance 1e-3 and a 50,000 iteration cap. Multi-class is handled with the default one-against-rest strategy. LinearSVC was preferred over a kernel SVM because the concatenated TF-IDF feature space is already very high-dimensional and sparse, so a linear decision boundary is usually enough on text [4]. Compared to Naive Bayes, the margin objective does not assume feature independence, so it can take advantage of the correlated n-gram features that NB cannot. The full pipeline is:

```
FeatureUnion(word_tfidf, char_tfidf, text_stats)
-> MaxAbsScaler
-> LinearSVC(C=1.0)
```

4 Setup

Datasets. Two datasets are used. The first is a synthetic corpus of 3,000 reports generated from a fixed template library (six sentence templates per class plus shared vocabulary lists for actions, components, errors, closing sentences and OS versions); the four classes hardware_bug, os_bug, platform_bug and ui_bug are exactly balanced at 750 reports each. The second is the computer-categories subset of 20 Newsgroups, fetched through sklearn.datasets.fetch_20newsgroups with the four categories comp.sys.ibm.pc.hardware (982), comp.os.ms-windows.misc (985), comp.sys.mac.hardware (963) and comp.windows.x (988), giving 3,918 real posts that map naturally to the same four bug-class labels. Headers, footers and quoted replies are stripped on download. A single seed (42) is used for the synthetic shuffle, both 10-fold splits, the 80/20 splits and inside LinearSVC, so the entire run is deterministic.

Baseline. Multinomial Naive Bayes with unigram TF-IDF (max_features=50,000, English stop-word removal, min_df=2, alpha=1.0). This is the Lab 1 setup, so the comparison is like for like on both datasets.

Metrics. Accuracy is the obvious metric and the one used in most of the lecture examples. I also report Macro-averaged F1, which weights the per-class F1 scores equally so a class with poor recall cannot hide behind a high overall accuracy, and the Matthews Correlation Coefficient [7], which takes all cells of the confusion matrix into account and tends to be more honest than F1 on multi-class problems.

Protocol. Stratified 10-fold cross-validation with seed 42 is the main protocol; stratification keeps the same class proportions in every fold. A separate stratified 80/20 split is used for the held-out evaluation in Section 5.3. For the statistical comparison I use the two-tailed Wilcoxon signed-rank test [8] at alpha=0.05, applied to the 10 paired per-fold scores. Wilcoxon was the test covered in the evaluation lecture; it is non-parametric, paired and works on small samples, which fits 10 folds well [9].

Environment. Apple M-series laptop CPU, 16 GB RAM, Python 3.13, scikit-learn 1.6, scipy 1.15. The full run on both datasets (data preparation, both classifiers, 10-fold CV, Wilcoxon, held-out splits, the feature contribution check on the synthetic corpus and all plots) takes about four minutes; no GPU is needed.

5 Experiments

5.1 Cross-Validation Results

Table 2 reports the mean and standard deviation of each metric across the 10 stratified folds, on both datasets.

Dataset	Approach	Accuracy	Macro-F1	MCC
Synthetic	Baseline (NB + TF-IDF)	0.9793 +/- 0.0083	0.9793 +/- 0.0084	0.9726 +/- 0.0110
Synthetic	Solution (SVC + enriched)	1.0000 +/- 0.0000	1.0000 +/- 0.0000	1.0000 +/- 0.0000
20 News.	Baseline (NB + TF-IDF)	0.8068 +/- 0.0110	0.8058 +/- 0.0111	0.7438 +/- 0.0145
20 News.	Solution (SVC + enriched)	0.8083 +/- 0.0188	0.8080 +/- 0.0184	0.7450 +/- 0.0251

Table 2. 10-fold cross-validation results on both datasets.

On synthetic the solution wins every fold: roughly +0.02 on Accuracy and Macro-F1, +0.027 on MCC. NB at 97.9% is already strong because the four classes use distinct vocabulary. On 20 Newsgroups both classifiers drop to about 80% Accuracy due to mixed vocabulary, off-topic content and overlapping technical jargon; the solution edges ahead in the means (+0.0015 Accuracy, +0.0022 Macro-F1, +0.0012 MCC) but the gap is small relative to fold-to-fold variance.

5.2 Statistical Significance

Dataset	Metric	W statistic	p-value	Significant (p<0.05)?
Synthetic	Accuracy	0.00	0.0020	Yes
Synthetic	Macro-F1	0.00	0.0020	Yes
Synthetic	MCC	0.00	0.0020	Yes
20 News.	Accuracy	20.00	0.7969	No
20 News.	Macro-F1	26.00	0.9219	No
20 News.	MCC	26.00	0.9219	No

Table 3. Wilcoxon signed-rank test, two-tailed, alpha=0.05.

On synthetic all three metrics are significant: W=0 is the smallest value the test can return and p=0.0020 the smallest two-tailed p-value reachable with 10 folds, so the test is at its tightest possible setting. On 20 Newsgroups the per-fold scores split roughly evenly, the W statistics sit near the centre and the test cannot reject the null at any sensible level. On noisy real text the enriched features and the NB baseline are statistically equivalent under this protocol.

5.3 Held-Out Test Set

Table 4 reports the held-out 80/20 results on both datasets. The synthetic split mirrors the CV story: the baseline scores 0.977/0.977/0.969 and the solution scores 1.000 on every metric. On 20 Newsgroups both classifiers are around 0.80 and within a percentage point of each other. The full classification reports are printed by src/main.py and the per-cell confusion matrices are saved to results/cm_synthetic_*.png and results/cm_20news_*.png.

Dataset	Model	Accuracy	Macro-F1	MCC
Synthetic	Baseline	0.9767	0.9766	0.9690
Synthetic	Solution	1.0000	1.0000	1.0000
20 News.	Baseline	0.8061	0.8054	0.7429
20 News.	Solution	0.7959	0.7965	0.7282

Table 4. Held-out 80/20 evaluation on both datasets.

On the synthetic split the baseline mostly loses recall on hardware_bug (0.94 in the per-class report); hardware reports share vocabulary with OS reports ("driver", "kernel", "module") and the unigram bag-of-words cannot separate them. The character n-grams in the solution capture more class-specific fragments such as "AHCI", "BIOS" and "ACPI", which closes the remaining gap. On 20 Newsgroups the two classifiers actually swap positions on the held-out split (baseline ahead by 0.01) which is exactly what we expect when the per-fold results are statistically tied.

5.4 Feature contribution check

On the full 80/20 synthetic split every variant of the solution scores 1.000, so the comparison is not informative there. I re-ran the same five variants with a much smaller training set: 25 reports per class (100 in total) for training and the remaining 2,900 reports for testing, with everything else unchanged. Table 5 reports the resulting scores.

Configuration	Accuracy	Macro-F1	MCC
Full solution (word + char + stats)	0.9566	0.9567	0.9421
No character n-grams	0.9362	0.9359	0.9152
No text statistics	0.9555	0.9557	0.9408
Word unigrams only (drop bigrams)	0.9548	0.9550	0.9399
Word unigrams only, no char, no stats	0.9400	0.9399	0.9202

Table 5. Feature contribution check on the synthetic corpus, train=100 / test=2,900.

Character n-grams are the single biggest contributor: dropping them costs 0.020 Accuracy and 0.027 MCC. Removing word bigrams (the unigrams-only row) only costs 0.002 at this training size, which suggests bigrams add little when there are too few examples to estimate their TF-IDF weights well; their value shows up at full data scale, where the full pipeline reaches 1.000 while the unigram-only NB baseline in Section 5.1 stops at 0.9793. The text-statistics block adds only 0.001 to Accuracy on this corpus but costs almost no compute, so it is kept in case it helps on noisier inputs. The fact that character n-grams help on synthetic but not enough to win on 20 Newsgroups (Section 5.1) lines up with this finding: the noisy real posts already cover so many surface variations that the n-gram-shaped signal NB misses on the synthetic corpus is largely covered by the unigram bag-of-words on real text.

5.5 Training and prediction time

Held-out timings (Table 6) were measured with time.perf_counter on a single thread. The baseline is roughly 30x faster on synthetic and 90x faster on 20 Newsgroups, but in absolute terms the solution still finishes well inside a second on synthetic and around 21 seconds on the larger 20 News. split, which would be acceptable for periodic retraining of a triage tool.

Dataset	Model	Fit time (s)	Predict time (s)
Synthetic	Baseline	0.014	0.004
Synthetic	Solution	0.489	0.069
20 News.	Baseline	0.226	0.054
20 News.	Solution	21.189	0.999

Table 6. Fit and predict times on the held-out 80/20 split.

5.6 Visualisation

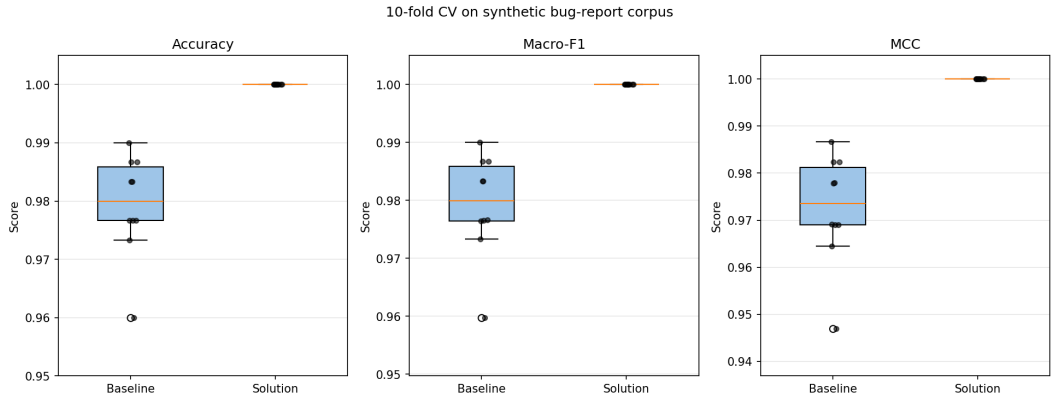

Figure 1. 10-fold CV scores on the synthetic corpus. Baseline (blue) clusters around 0.97-0.99; solution (green) sits at exactly 1.000. The corresponding 20 Newsgroups boxplot in results/boxplot_20news.png shows the two boxes sitting on top of each other.

6 Reflection

What the two datasets together say. The two-dataset comparison gives a more honest picture than either dataset alone would. The synthetic corpus shows that the enriched feature set can beat the NB baseline by a clear and statistically significant margin when the underlying class structure is clean. The 20 Newsgroups results show that this advantage shrinks to within fold-to-fold noise on real noisy text. Two things plausibly drive the gap. First, real posts already cover so many surface variations of the same words that the sub-word patterns character n-grams pick up on synthetic data are largely redundant. Second, 20 Newsgroups posts are much longer and contain off-topic content (jokes, signatures, quoted replies that survived the cleanup) that adds noise the small text-stats block was never designed to handle. So the headline 1.000 on synthetic should be read as an upper bound: the method works, but the real-world ceiling is around 80% with the current features.

Limitations. On the synthetic corpus the baseline already hits 97.9%, which means there is little room left to improve and the four classes use distinct enough vocabulary that the unigram bag-of-words is almost sufficient. On 20 Newsgroups the classifier still ignores the metadata that real bug trackers carry (reporter history, product component, version, OS field) which is one of the main contributors to the gains DRONE reports [3]. The character n-gram block also adds about 30,000 dimensions, which on a tracker with millions of reports would start to hurt memory and would justify either feature selection or a truncated SVD step. Everything is tied to a single random seed (42); a fuller study would repeat the whole pipeline over a sweep of seeds to get tighter confidence intervals.

Possible improvements. In priority order: (i) fine-tune a pre-trained model such as BERT [5] on real bug-report data; this is the change most likely to push real-world accuracy beyond the 80% ceiling seen on 20 Newsgroups; (ii) add the structured metadata that real trackers expose as further FeatureUnion blocks; this is cheap and aligns with what DRONE [3] showed for priority prediction; (iii) try cross-project transfer (training on Eclipse, evaluating on Mozilla) to check whether the method generalises beyond a single project vocabulary.

7 Conclusion

I built a Linear SVM classifier for bug reports that combines word bigrams, character n-grams and a small set of hand-picked text statistics on top of TF-IDF, and compared it to the Naive Bayes + TF-IDF baseline from Lab 1 on two datasets. On the synthetic 3,000-report corpus the solution reached 1.000 on Accuracy, Macro-F1 and MCC against 0.979/0.979/0.973 for the baseline; the improvement is significant on all three metrics under a Wilcoxon signed-rank test (p=0.002, the smallest possible value with 10 folds). On the 20 Newsgroups computer-categories subset both classifiers land near 0.81 Accuracy and the difference is not statistically significant (p between 0.79 and 0.92). The feature contribution check in Section 5.4 isolates character n-grams as the biggest single contributor on

low-resource synthetic data (Accuracy drops by 0.020 when they are removed); on 20 Newsgroups the same features add little because the real posts already cover the surface variations they were designed to capture. The solution stays CPU-only on both datasets, finishing under a second on synthetic and around 21 seconds on 20 Newsgroups. Code, raw CSVs, figures and instructions to reproduce everything from a single command are in the public artifact.

8 Artifact

Source code, raw results and figures are at: https://github.com/git-frenzy/ISE-Bug-Classification. The repository root contains requirements.docx, manual.docx and replication.docx (export to PDF for the marker bundle if needed).

References

- [1] J. Anvik, L. Hiew, and G. C. Murphy, "Who should fix this bug?" in Proc. ICSE, 2006, pp. 361-370.
- [2] N. Limsettho, K. Hotta, Y. Higo and S. Kusumoto, "Comparing and combining heuristic approaches for bug report classification," in Proc. APSEC, 2014, pp. 279-286.
- [3] Y. Tian, C. Sun and D. Lo, "DRONE: Predicting priority of reported bugs by multi-factor analysis," in Proc. ICSM, 2012, pp. 200-209.
- [4] T. Joachims, "Text categorization with support vector machines: Learning with many relevant features," in Proc. ECML, 1998, pp. 137-142.
- [5] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, 2019, pp. 4171-4186.
- [6] X. Zhang, J. Zhao and Y. LeCun, "Character-level convolutional networks for text classification," in Advances in NeurIPS, 2015, pp. 649-657.
- [7] D. Chicco and G. Jurman, "The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation," BMC Genomics, vol. 21, no. 6, 2020.
- [8] F. Wilcoxon, "Individual comparisons by ranking methods," Biometrics Bulletin, vol. 1, no. 6, pp. 80-83, 1945.
- [9] A. Arcuri and L. Briand, "A practical guide for using statistical tests to assess randomized algorithms in software engineering," in Proc. ICSE, 2011, pp. 1-10.
- [10] J. Pennington, R. Socher and C. D. Manning, "GloVe: Global vectors for word representation," in Proc. EMNLP, 2014, pp. 1532-1543.
- [11] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado and J. Dean, "Distributed representations of words and phrases and their compositionality," in NeurIPS, 2013, pp. 3111-3119.
- [12] A. N. Lam, A. T. Nguyen, H. A. Nguyen and T. N. Nguyen, "Bug localization with combination of deep learning and information retrieval," in Proc. ICPC, 2017, pp. 218-229.
- [13] G. Antoniol, K. Ayari, M. Di Penta, F. Khomh and Y.-G. Gueheneuc, "Is it a bug or an enhancement? A text-based approach to classify change requests," in CASCON, 2008, pp. 304-318.